SilkMind: Software and Symbolic AI, Learning Content V1.1

Team: SilkMind
Repository: https://github.com/skh0035/Mini-AI-repository
Rendered: 31 October 2026 from the `content/` folder
Contents: Module outline and development status · Topics 1–5

Module Outline: Software & Symbolic AI

Team: SilkMind | **Version:** Markdown Content V1.1 (revised after Stage 1 feedback) | **Last updated:** 31 October 2026

This outline is the map of the whole module. It shows what learners will be able to do, the order in which topics are taught and why, what each topic contains, and how far each section has been developed.

---

1. Who this module is for

Intended learners: first-year university students from any subject who have **no prior experience** of programming, computer science or artificial intelligence.

What we assume learners can already do (prerequisites):

Prerequisite	Why it is needed	Where it is covered if missing
Use a computer or phone: open apps, log in, browse the web	All examples are built on everyday apps (login screens, thermostats, spam filters)	Not covered; assumed
Read a simple table	Truth tables and trace tables are used from Topic 1 onwards	Topic 1 explains how to read a truth table
Basic comparisons with numbers (`<`, `>`, `=`)	Rules such as `T < 18`	Topic 1, Section 3
Everyday reasoning with "if … then …"	All rules are written in IF–THEN form	Built up in Topic 1

Not required: any programming language, maths beyond basic comparisons, or knowledge of how AI models are trained. The short Python demo in Topic 2 is optional.

---

2. Intended learning outcomes

By the end of the module, learners should be able to:

#	Learning outcome	Taught in
LO1	Explain what software, an algorithm and a Boolean condition are, using an everyday example.	Topic 1
LO2	Write a simple decision as an IF–THEN rule and check it with a truth table.	Topic 1
LO3	Describe the parts of a rule-based (symbolic) AI system: facts, rules, knowledge base and inference engine.	Topic 2
LO4	Trace forward chaining by hand to show how a rule-based system reaches a conclusion and explains it.	Topic 2
LO5	Contrast symbolic AI with machine learning and choose a suitable approach for a given problem.	Topic 3
LO6	Explain why symbolic AI rose, fell (the "AI winters") and changed, using key events as evidence.	Topic 4
LO7	Identify where rule-based reasoning is used in modern software, including systems that combine rules with machine learning.	Topic 5

3. Learning sequence and why it is in this order

```
Topic 1  Software & basic logic        "Computers follow strict true/false rules"
   │
   ▼
Topic 2  Rules & inference             "Symbolic AI = many such rules + an engine that
chains them"
   │
   ▼
Topic 3  Symbolic AI vs machine learning   "Writing the rules yourself vs learning them
from data"
   │
   ▼
Topic 4  History: boom, AI winters, lessons   "Why the rules-only approach hit its
limits"
   │
   ▼
Topic 5  Symbolic AI in modern software        "Where rules are still used today, alone
or with ML"
```

Why this order:

- Topic 1 gives the one idea everything else depends on: a computer decides by testing true/false conditions.
- Topic 2 scales that idea up: a symbolic AI system is a large collection of IF–THEN rules plus a program that applies them automatically.
- Topic 3 introduces the other main approach (machine learning) only **after** learners understand rules, so the comparison has something concrete to compare.

- Topic 4 (history) comes after Topics 2–3 on purpose. The AI winters are much easier to understand once learners have seen a rule-based system and know what its limits are. *(In V1.0, history was Topic 2. We moved it, because learners would have been studying the collapse of something they had not yet met.)*
- Topic 5 closes the module by connecting everything to software learners use today.

---

4. Topic-by-topic plan and development status

Status labels used:

Label	Meaning
Outline only	Headings and plan, no learning text yet
Drafted	Explanations, example and activity written; not yet reviewed by a second member
Checked	Reviewed by a second member; facts checked against sources; corrections made
Needs revision	Reviewed; problems found that are not yet fixed

> **Honest note on status (31 Oct 2026):** the V1.1 text of Topics 2–5 and the revision of Topic 1 were drafted with AI assistance (see `docs/ai-use-log.md`). No section is labelled **Checked** until a team member has completed the review in `docs/review-log.md`.

Topic 1: Introduction to Software and Basic Logic

File: `01-software-and-logic.md` | **Outcomes:** LO1, LO2 | **Est. time:** 30 min

Subtopic	Planned example / activity	Status
1.1 Software vs hardware	Quick-check table: sort 6 items into hardware/software	Drafted
1.2 Algorithms	Recipe comparison; login check as an algorithm	Drafted
1.3 Boolean conditions and operators (AND, OR, NOT)	Operator table with everyday examples	Drafted
1.4 Worked example: login decision	Step-by-step trace, truth table, pseudocode, flowchart	Drafted (V1.0 content, revised)
1.5 Learner activity: thermostat rule	Write a 3-step algorithm with one IF–THEN–ELSE rule; stretch task with AND	Drafted (activity wording fixed)
1.6 Check your understanding	4 short questions with answers	Drafted

Topic 2: Rules and Inference: How a Symbolic AI Reasons ★ *fully developed topic*

File: `02-rules-and-inference.md` | **Outcomes:** LO3, LO4 | **Est. time:** 45 min

Subtopic	Planned example / activity	Status
2.1 What "symbolic" means	Symbols as labels; why the computer doesn't "understand" them	Drafted
2.2 Facts, rules, knowledge base, working memory	Animal-identifier knowledge base (6 rules)	Drafted
2.3 The inference engine and forward chaining	Worked example with cycle-by-cycle trace table + diagram	Drafted
2.4 Backward chaining (goal-driven)	Same knowledge base, asking "Is it a penguin?"	Drafted
2.5 Explanations ("why?")	How the system shows the rules that led to its answer	Drafted
2.6 Real expert systems and their limits	DENDRAL, MYCIN, XCON; the MYCIN gunshot-wound limitation	Drafted
2.7 Learner activity + answers	4-part activity: trace, extend, backward chain, find the gap	Drafted
2.8 Optional code demo	`examples/topic02_mini_rule_engine.py` (prints the same trace)	Drafted; code tested

Topic 3: Symbolic AI vs Machine Learning

File: `03-symbolic-ai-vs-machine-learning.md` | **Outcome:** LO5 | **Est. time:** 30 min

Subtopic	Planned example / activity	Status
3.1 Two ways to give a computer knowledge: write rules vs learn from examples	Diagram: rules + data → answers vs data + answers → rules	Drafted
3.2 Same problem, two approaches: a spam filter	Rule-based filter vs a filter trained on labelled emails	Drafted
3.3 Comparison table: explainability, data needed, flexibility, maintenance	Table	Drafted
3.4 Learner activity: choose an approach for 5 scenarios	Answers with reasons	Drafted
3.5 How machine learning actually trains a model	*Deliberately brief; beyond this module's scope*	Outline only

Topic 4: History: The Rise, the AI Winters and the Lessons

File: `04-history-and-ai-winters.md` | **Outcome:** LO6 | **Est. time:** 25 min

Subtopic	Planned example / activity	Status
4.1 1956: the Dartmouth workshop and early optimism	Timeline	Drafted
4.2 First AI winter (≈1974–1980): Lighthill report, funding cuts	Cause → effect table	Drafted
4.3 The expert-system boom of the 1980s	XCON as a case study	Drafted
4.4 Second AI winter (≈1987–1993): brittleness, maintenance cost, LISP machine collapse	Link back to the limits seen in Topic 2	Drafted
4.5 Learner activity: match each event to its cause	Answers	Drafted
4.6 Timeline graphic for slides	Visual design lead to produce	Outline only

Topic 5: Symbolic AI in Modern Software

File: `05-symbolic-ai-in-modern-software.md` | **Outcome:** LO7 | **Est. time:** 25 min

Subtopic	Planned example / activity	Status
5.1 Rules hidden in everyday software (eligibility checks, forms, game characters)	Annotated examples	Drafted
5.2 Knowledge graphs	Small fact-network example	Drafted
5.3 Neuro-symbolic AI: rules + learning together	AlphaGeometry (2024) case study	Drafted
5.4 Learner activity: spot the rules in an app you use	Expected-outcome guide	Drafted
5.5 Module wrap-up and self-assessment quiz	8-question quiz covering LO1–LO7	Outline only

---

5. Known gaps and uncertainties

We list these openly so that reviewers and the teacher can see what is still weak.

1. **No section is "Checked" yet.** A second team member must review each topic using `docs/review-log.md`.
2. **Topic 5 sources are thinner than Topics 1–2.** Section 5.1 describes general software practice (business rules, game AI) with few direct sources. We need one reliable source per example or must cut the example.
3. **Topic 4 dates are approximate.** Historians give slightly different start and end years for the AI winters. We use the commonly cited ranges and say "about".
4. **Topic 3 simplifies machine learning heavily.** We do not explain how training works. We need to check that the simplification does not become misleading.

5. **Uncertainty handling is not covered.** Real expert systems such as MYCIN used confidence values. We mention this only briefly in Topic 2; a short optional box could be added.
6. **Module quiz (5.5) and Topic 4 timeline graphic are not yet started.**

---

6. From Markdown to the three learning resources

Resource	How this Markdown will be used	First section to convert
Study Guide	Each topic file becomes one chapter. Learning objectives, key terms, worked example, activity and answers are kept; answers move to an appendix so learners try first.	Topic 2
PPT Slides	One slide per subtopic. Diagrams (login flowchart, expert-system architecture, forward-chaining trace) become the main slide visuals; trace tables are revealed one row per click.	Topic 2, Section 3 (forward-chaining trace)
Video Tutorial	A 4–5 minute screen recording that walks through the forward-chaining trace by hand, then runs `examples/topic02_mini_rule_engine.py` to show the computer producing the same trace.	Topic 2, Sections 3 and 8

Topic 1: Introduction to Software and Basic Logic

Level: Beginner | **Estimated time:** 30 minutes | **Status:** Drafted (V1.0 content, revised 31 Oct 2026) **Comes before:** Topic 2: Rules and Inference

Learning objectives. After this topic you will be able to:

1. Explain what software is and how it differs from hardware.
2. Define an algorithm and a Boolean condition in your own words.
3. Trace a simple decision (a login check) step by step and check it with a truth table.
4. Write a simple decision as an IF–THEN rule.

---

1. Why start here?

The rest of this module is about **Symbolic AI**: AI that works by following rules that people write down. To understand how such a system "thinks", you first need to understand how *any* program makes a decision. This topic gives you that foundation in about half an hour.

---

2. Concept explanation

2.1 Software and hardware

A computer is a machine. On its own, it does nothing useful. It needs instructions, and those instructions are called **software**.

> **Software:** the instructions (programs) that tell a computer what to do. The word was coined to distinguish these instructions from **hardware**, the physical parts of a computer (Britannica: Software).

You cannot touch software. You *can* touch hardware: the screen, the processor chip, the battery, the keyboard.

Hardware (physical)	Software (instructions)
Keyboard, screen, processor, memory chip	Operating system (e.g. Windows, Android)
Temperature sensor in a thermostat	The thermostat's control program
Phone camera	The camera app

Two common types of software are **system software**, which runs the computer itself (such as an operating system), and **application software**, which does a task for the user (such as a word processor or email app) (BBC Bitesize).

2.2 Computers follow instructions exactly

Computers cannot guess what you meant. They follow instructions exactly, one step at a time, in the order given. If an instruction is vague ("turn the heater on if it's a bit cold"), the computer has no way to ask what "a bit cold" means.

So programmers write instructions as precise, step-by-step procedures.

> **Algorithm:** a step-by-step procedure that solves a problem in a finite number of steps (Britannica: Algorithm). Each step must be clearly defined, and the procedure must eventually stop.

A recipe is a good everyday comparison: a list of clear steps, in order, with a finished dish at the end. A **program** is an algorithm written in a form a computer can run.

2.3 Decisions: true or false

Many steps in an algorithm are **decisions**, for example:

> IF the password is correct, THEN let the user in. OTHERWISE, refuse.

The question "is the password correct?" has only two possible answers: **true** or **false**. A question like this is called a **condition** (or **Boolean condition**).

> **Boolean logic:** a system of logic in which every statement is either true (1) or false (0), and statements are combined with the operators AND, OR and NOT. It is named after the English mathematician George Boole, who first set it out in 1847 (Britannica: Boolean algebra).

Operator	Meaning	Everyday example	True when…
AND	Both parts must be true	"Account exists" AND "password matches"	both are true
OR	At least one part must be true	"It is raining" OR "it is snowing"	either one (or both) is true
NOT	Flips the answer	NOT "account is locked"	the account is **not** locked

> **Truth table:** a table that lists every possible combination of true/false inputs and shows the result for each. It lets you check a rule completely, with no case forgotten (EBSCO Research Starters: Boolean algebra).

2.4 Why this matters for AI

The earliest approach to artificial intelligence, **Symbolic AI**, is built from exactly these pieces. Experts write their knowledge down as many IF–THEN rules, and a program applies those rules automatically (Britannica: Expert systems). If you understand how one program makes one decision, you already understand the building block of a rule-based AI. Topic 2 shows how many rules are combined.

Key idea: Software = precise instructions. Many instructions are true/false decisions. Rule-based AI is built from the same kind of decisions.

---

3. Worked example: how does an email app decide whether you may log in?

Input (what the program receives): an email address and a password typed by the user.
Output (what the program produces): either "open the inbox" or an error message.

Step 1: Does an account exist for this email? The program looks up the email address in its list of accounts.

- Found → the condition is **true**. Continue to Step 2.
- Not found → the condition is **false**. Refuse access.

Step 2: Does the password match? The program compares the typed password with the one stored for that account.

- Match → **true**. Continue to Step 3.
- No match → **false**. Refuse access.

Note: in well-built systems, passwords are not stored as plain text. They are stored in a scrambled form ("salted and hashed"), and the program compares scrambled versions (NIST SP 800-63B-4). The true/false logic is the same.

Step 3: Decide and respond.

- If Step 1 AND Step 2 were both true → grant access and open the inbox.
- Otherwise → deny access.

The whole decision as one Boolean rule:

```
access_granted = (account_exists) AND (password_matches)
```

Checking the rule with a truth table:

Account exists?	Password matches?	`account_exists AND password_matches`	Result
True	True	True	**Access granted**
True	False	False	Access denied
False	True*	False	Access denied
False	False	False	Access denied

*In practice a password cannot match an account that does not exist. The row is included because a truth table always shows every combination. Only one row out of four leads to "granted", which is exactly how AND behaves.

The same logic written as pseudocode.

> **Pseudocode:** program-like steps written in plain language. It is not a real programming language; it is a way to write an algorithm clearly before (or instead of) coding it.

```
IF account_exists THEN
    IF password_matches THEN
        show inbox
    ELSE
        show "Invalid email or password"
ELSE
    show "Invalid email or password"
```

A design detail worth noticing. Both failure paths show the *same* message. This is deliberate. Security guidance recommends one generic error message whether the email or the password was wrong, so that attackers cannot use the login page to discover which email addresses have accounts (OWASP Authentication Cheat Sheet). Even this small program is a set of carefully chosen rules.

---

4. Visual: the login decision as a flowchart

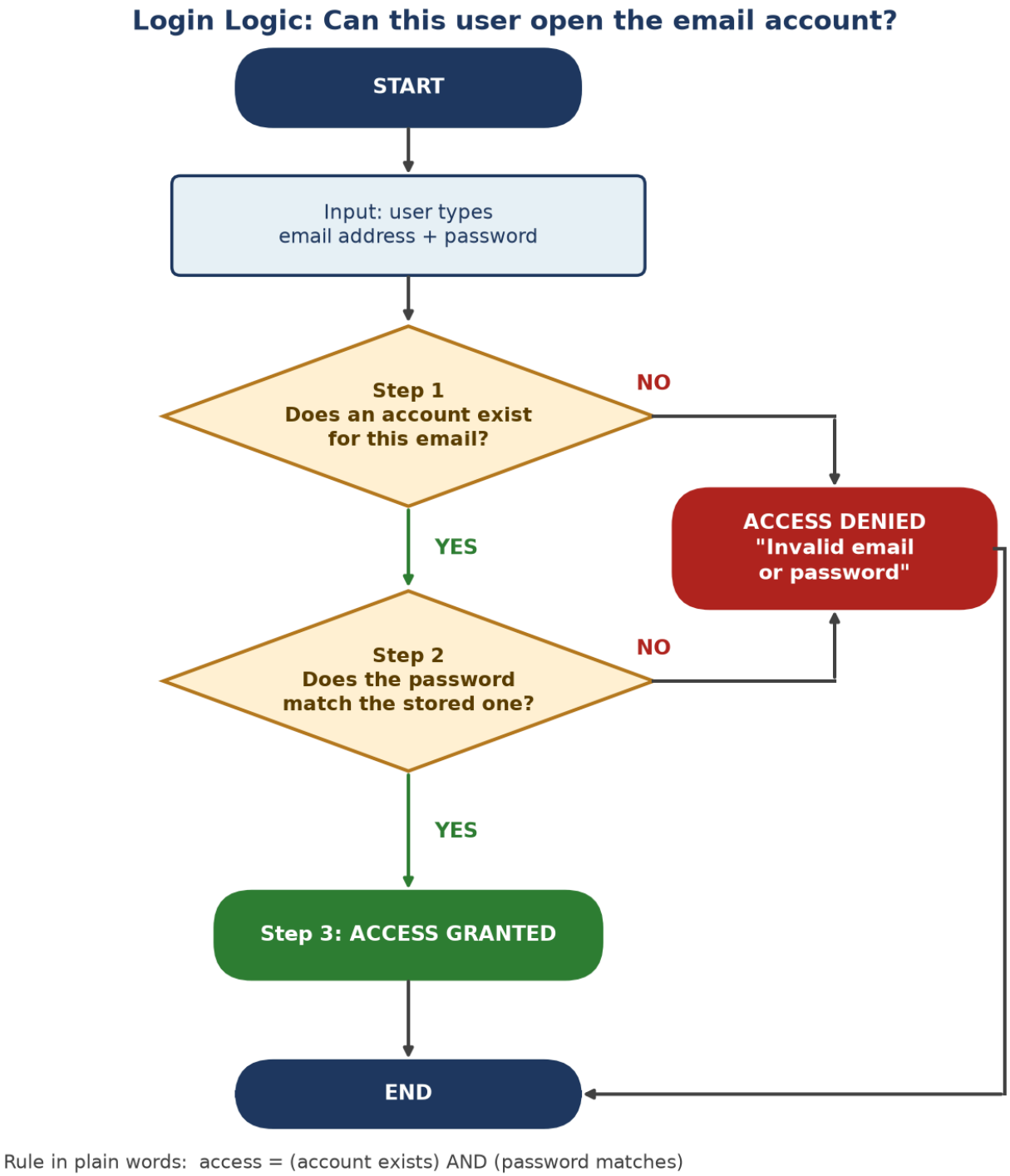


Figure 1.1. The login algorithm as a flowchart (added to the repository by @llex-27). Start at the top and follow the arrows. Rounded boxes are the start, end and outcomes; the rectangle is the input; each diamond is one Boolean test with exactly two exits (YES/NO). The only path to "Access granted" answers YES to both questions.

---

5. Learner activity: a thermostat rule

Scenario. You are writing the software for a smart thermostat. It has a temperature sensor and can switch a heater ON or OFF.

Task A (main task). Write a **3-step algorithm** that turns the heater on when the temperature is below 18 °C. Use the same pattern as the login example:

1. Get the input.
2. Test a true/false condition.
3. Act on the result, using **one IF–THEN–ELSE rule**.

Task B (truth table). Fill in the result column for three temperature readings: 15 °C, 18 °C and 21 °C.

Task C (stretch). Change your rule so the heater turns on only if the temperature is below 18 °C **and** someone is at home. Which Boolean operator did you use? Write the truth table for the new condition.

Hints: "below 18 °C" is a comparison, so it is either true or false for any reading. Decide what the heater should do when the condition is false. Write your answers before reading Section 6.

---

6. Expected outcome and answers

Task A: model answer

Step	What the software does	Matching step in the login example
1. Get the input	Read the current temperature from the sensor and call it `T`.	The user types an email and password.
2. Test the condition	Evaluate `T < 18`. The result is **true** or **false**.	"Does the account exist?" / "Does the password match?"
3. Act on the result	IF `T < 18` is true, THEN switch the heater **ON**; ELSE switch it **OFF**.	Grant or deny access.

```
T = read temperature sensor
IF T < 18 THEN
    heater = ON
ELSE
    heater = OFF
```

Why this is correct:

- It is an **algorithm**: a short, ordered list of clear steps that finishes with a result. A real thermostat would repeat it every few seconds.
- Step 2 is a **Boolean condition**: `T < 18` can only be true or false, so the program always knows what to do next.
- Step 3 is an **IF–THEN–ELSE rule**. It covers both outcomes, so the heater is never left in an unknown state.

Task B: truth table

Reading `T`	`T < 18`	Heater
15 °C	True	ON
18 °C	**False**	OFF
21 °C	False	OFF

The 18 °C row is the one most people get wrong. "Below 18" means *strictly less than* 18, so at exactly 18 °C the condition is false and the heater stays off. If you wrote `T <= 18`, you changed the rule. Computers need this kind of precision.

Task C: stretch answer

Operator: **AND**. Rule: `IF (T < 18) AND (someone_home) THEN heater = ON ELSE heater = OFF`.

`T < 18`	`someone_home`	Heater
True	True	**ON**
True	False	OFF
False	True	OFF
False	False	OFF

Common mistakes to check for

1. **Skipping the input step.** The program must read the temperature before it can compare it.
2. **Writing a vague rule** such as "if it's cold, heat it". "Cold" is not a defined true/false condition.
3. **Forgetting the ELSE.** Without it, nothing says what happens when the condition is false.

---

7. Check your understanding

1. Is a phone's camera app hardware or software? What about the camera lens?
2. Why is "turn the heater on if it's chilly" *not* a good instruction for a computer?
3. `has_ticket = True`, `ticket_expired = True`. What is the result of `has_ticket AND NOT ticket_expired`?
4. A café gives a discount to customers who are students **or** over 65. Which operator is needed, and does a 70-year-old student get the discount?

Answers.

1. The app is software; the lens is hardware.

2. "Chilly" has no exact true/false meaning, so the computer cannot test it. A precise condition such as `T < 18` is needed.
3. `NOT ticket_expired` is False, so `True AND False` = **False**: the person cannot travel.
4. **OR**. Yes: OR is true when at least one part is true, and here both are.

---

8. Summary

- **Software** is the set of instructions; **hardware** is the physical machine.
- An **algorithm** is a finite list of clear steps; a program is an algorithm a computer can run.
- Decisions inside algorithms are **Boolean conditions** (true/false), combined with **AND**, **OR** and **NOT**, and checked with **truth tables**.
- An **IF–THEN rule** is the building block of rule-based (Symbolic) AI, which Topic 2 explains.

---

9. References

All links accessed on 21 October 2026 and re-checked on 31 October 2026.

1. Encyclopaedia Britannica. *Software.* https://www.britannica.com/technology/software
2. BBC Bitesize. *What is software?* (Computational constructs, National 4). https://www.bbc.co.uk/bitesize/guides/zcg9kqt/revision/1
3. Encyclopaedia Britannica. *Algorithm.* https://www.britannica.com/science/algorithm
4. Encyclopaedia Britannica. *Boolean algebra.* https://www.britannica.com/topic/Boolean-algebra
5. EBSCO Research Starters. *Boolean algebra.* https://www.ebsco.com/research-starters/science/boolean-algebra
6. Encyclopaedia Britannica. *Artificial intelligence: Expert systems.* https://www.britannica.com/technology/artificial-intelligence/Expert-systems
7. Temoshok, D., et al. (2025). *NIST Special Publication 800-63B-4, Digital Identity Guidelines: Authentication and Authenticator Management.* National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63B-4.pdf
8. OWASP Foundation. *Authentication Cheat Sheet.* https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Changes from V1.0 (for reviewers): activity wording now matches the model answer (V1.0 asked for "3 rules" but answered with 3 steps); terms *condition*, *truth table* and *pseudocode* are now defined where they first appear; added a hardware/software table, a truth-table task and a self-check section; removed references that were listed but not used (Britannica Kids, Plus Maths, Stanford Python notes, Crash Course) and replaced a lecture-notes PDF and Wikipedia with Britannica's expert-systems article.

Topic 2: Rules and Inference: How a Symbolic AI Reasons

Level: Beginner | **Estimated time:** 45 minutes | **Status:** Drafted (fully developed topic; awaiting team review) **Builds on:** Topic 1: Software and Basic Logic | **Comes before:** Topic 3: Symbolic AI vs Machine Learning

Learning objectives. After this topic you will be able to:

1. Explain what "symbolic" means in Symbolic AI.
2. Name and describe the parts of a rule-based AI system: facts, rules, knowledge base, working memory and inference engine.
3. Trace **forward chaining** by hand, cycle by cycle, and say when it stops.
4. Explain how the system can answer "why?" by showing the rules it used.
5. Give one real example of an expert system and one important limitation.

---

1. From one rule to many

In Topic 1 a program made **one** decision with **one** rule:

```
IF T < 18 THEN heater = ON ELSE heater = OFF
```

Now imagine a doctor's knowledge about blood infections, written down as **hundreds** of rules like this. A program that stores those rules and applies them automatically to a new patient can give advice that sounds like an expert's. That is the idea behind **Symbolic AI** and its best-known product, the **expert system**.

> **Expert system:** a computer program that uses AI methods to solve problems in one specialised area, the kind of problem that normally needs a human expert (Britannica: Expert system).

---

2. What does "symbolic" mean?

> **Symbol:** a name or label that stands for something, for example `is a bird`, `has feathers`, `temperature_high`.

Symbolic AI represents knowledge as symbols that humans can read, and reasons by combining those symbols with rules.

One important point: **the computer does not know what the symbols mean.** To the program, `has feathers` is just a string of characters. It does not know that feathers are soft or

that birds have them. All the "meaning" comes from the rules people write. For example, the program does not know that `cannot fly` is the opposite of `can fly` unless a rule says so. Keep this in mind; it explains many of the limits in Section 7.

---

3. The parts of a rule-based system

Part	What it is	Everyday comparison
Fact	A statement taken as true for the current case, e.g. `has feathers`.	A clue in a detective story
Rule	An IF–THEN statement. The **IF part** (also called the *condition* or *antecedent*) is one or more facts joined by AND. The **THEN part** (*conclusion* or *consequent*) is a new fact.	"If the footprints are wet, the suspect came in from outside"
Knowledge base	The stored collection of rules for one subject area. It is written once, usually with the help of human experts.	The detective's experience
Working memory	The facts about *this particular case*: the starting facts plus any new facts the system works out.	The detective's notebook
Inference engine	The program that repeatedly finds rules whose IF part is true and adds their THEN part to working memory.	The detective's reasoning process

> **Inference:** working out a new fact from facts you already have. Britannica describes an expert system's two basic parts as a knowledge base and an **inference engine** that draws conclusions from it (Britannica: Expert systems).

> **A rule "fires"** when the inference engine finds that its IF part is true and adds its THEN part to working memory.

Figure 2.1 Parts of a rule-based (symbolic) AI system

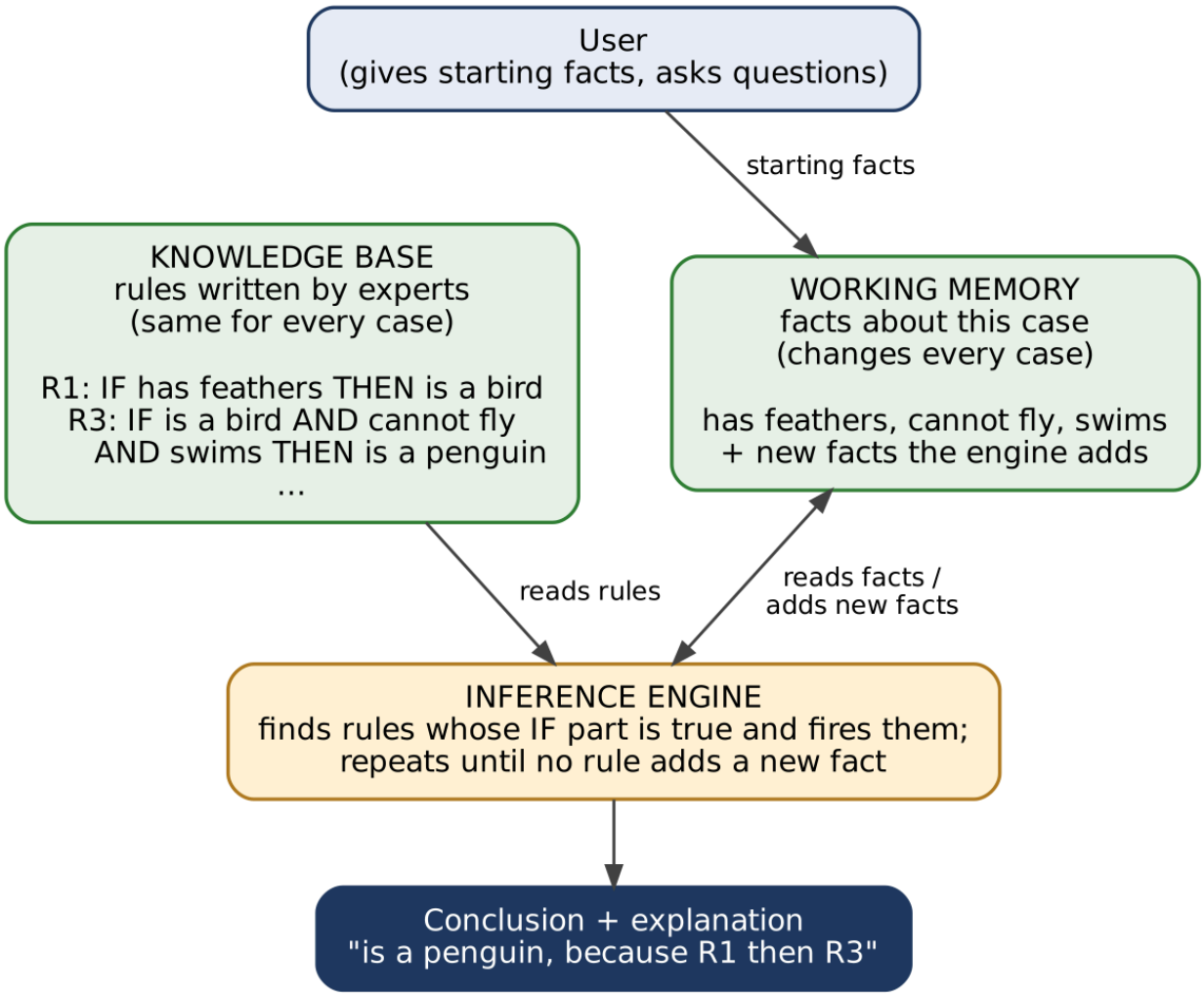


Figure 2.1. The parts of a rule-based system. The knowledge base stays the same for every case; working memory changes for each case. The inference engine is the only part that "does" anything. (Diagram source: `assets/diagram-sources/topic02-expert-system-architecture.dot`.)

---

4. Worked example: an animal identifier

We will build a tiny expert system that identifies an animal from a description. (Small animal-classification systems like this are a standard teaching example; see the frog example in Wikipedia: Forward chaining.)

4.1 The knowledge base (6 rules)

Rule	IF …	THEN …
R1	has feathers	is a bird
R2	gives milk	is a mammal
R3	is a bird AND cannot fly AND swims	is a penguin
R4	is a bird AND can fly AND hunts at night	is an owl
R5	is a mammal AND eats meat AND has black stripes	is a tiger
R6	is a mammal AND has hooves AND has black stripes	is a zebra

Notice that R3–R6 use facts that are produced by R1 and R2. Rules can build on each other. This is called **chaining**.

4.2 The case

A user describes an animal. The starting facts in working memory are:

```
has feathers,  cannot fly,  swims
```

4.3 Forward chaining, step by step

> **Forward chaining:** the inference engine starts from the known facts and keeps firing rules to add new facts, until no rule can add anything new. It is called **data-driven** because the facts decide which rules fire (Wikipedia: Forward chaining).

The engine works in **cycles**. In each cycle it checks the rules in order (R1, R2, …), fires the **first** rule whose IF part is completely true **and** whose THEN part is not already known, then starts the next cycle.

Cycle 1.

- R1: IF *has feathers*? Yes, it is in working memory → **R1 fires**. Add `is a bird`.

Cycle 2. Working memory now: `has feathers, cannot fly, swims, is a bird`.

- R1: IF part true, but `is a bird` is already known → skip (it would add nothing new).
- R2: *gives milk*? Not known → false.
- R3: *is a bird* ✔ AND *cannot fly* ✔ AND *swims* ✔ → **R3 fires**. Add `is a penguin`.

Cycle 3.

- R1, R3: true but add nothing new. R2, R4, R5, R6: at least one condition is not known → false.
- No rule can add a new fact → **STOP**.

Trace table (the standard way to record forward chaining):

Cycle	Working memory at start of cycle	Rules with a true IF part that would add something new	Rule fired	New fact
1	has feathers, cannot fly, swims	R1	**R1**	is a bird
2	… + is a bird	R3	**R3**	is a penguin
3	… + is a penguin	none	none	**stop**

Conclusion: the animal is a penguin.

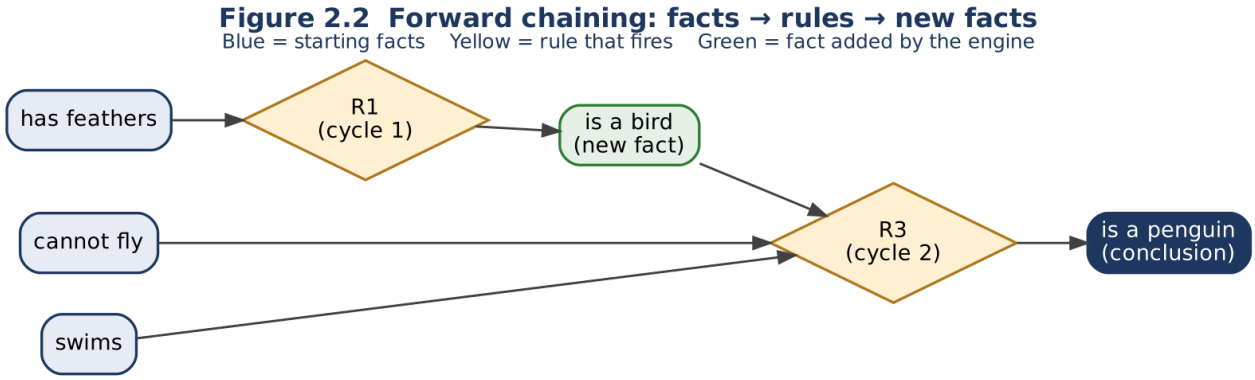


Figure 2.2. The same trace as a picture. Each rule takes facts in and produces one new fact; the new fact from cycle 1 becomes an input in cycle 2.

4.4 Asking "why?": explanations

Because every new fact was added by a named rule, the system can explain its answer by walking back through the chain:

```
Why "is a penguin"?   Because R3: is a bird AND cannot fly AND swims.
  Why "is a bird"?    Because R1: has feathers.
  "has feathers", "cannot fly", "swims" were given by the user.
```

This is one of the biggest strengths of Symbolic AI. Britannica notes that expert systems can show the sequence of rules behind a conclusion, which helps users judge whether to trust it (Britannica: Expert system). The MYCIN medical system, for example, could explain its reasoning on request (Britannica: Expert systems).

4.5 The same idea the other way round: backward chaining

> **Backward chaining:** the engine starts from a **goal** ("Is it a penguin?") and works backwards, checking which facts would be needed to prove it. It is called **goal-driven** (Wikipedia: Forward chaining).

Goal: *is a penguin?*

1. Which rule concludes `is a penguin`? **R3**. So we need: `is a bird`, `cannot fly`, `swims`.
2. `is a bird`: not given, but **R1** concludes it if `has feathers`. Is `has feathers` known? Yes ✔.
3. `cannot fly`: known ✔. `swims`: known ✔.
4. All conditions are proved → goal is **true**.

If a needed fact is not known, a backward-chaining system can **ask the user** ("Does the animal swim?"). This is how many diagnosis systems hold a conversation: they ask only the questions that matter for the goal they are testing.

	Forward chaining	**Backward chaining**
Starts from	Facts	A goal (a hypothesis)
Question it answers	"What can I conclude from these facts?"	"Is this particular conclusion true?"
Good for	Monitoring, configuration, "what follows?"	Diagnosis, "is it X?", asking targeted questions

4.6 Optional: watch a computer do it

The file `examples/topic02_mini_rule_engine.py` contains the same six rules and a 20-line inference engine. Running it prints exactly the trace above:

```
Starting facts: has feathers, cannot fly, swims

Cycle 1: R1 fires because has feathers -> add 'is a bird'
Cycle 2: R3 fires because is a bird AND cannot fly AND swims -> add 'is a penguin'
Cycle 3: no rule adds a new fact -> STOP

Why is it a penguin?
- 'is a penguin' because of R3 (IF is a bird AND cannot fly AND swims)
    - 'is a bird' because of R1 (IF has feathers)
        - 'has feathers' was given as a starting fact
    - 'cannot fly' was given as a starting fact
    - 'swims' was given as a starting fact
```

You do not need to read Python for this module. The point is that the computer's "reasoning" is nothing more than the loop you just did by hand.

---

5. Learner activity

Use the knowledge base in Section 4.1 (R1–R6) and the same strategy: in each cycle, fire the first rule (in order R1→R6) whose IF part is true and whose THEN part is new.

Part A: trace it. The starting facts are: `gives milk`, `has hooves`, `has black stripes`. Complete a trace table (cycle, working memory, rule fired, new fact). What does the system conclude, and in which cycle does it stop?

Part B: extend the knowledge base. Write one new rule, R7, that identifies a **giraffe**, using `is a mammal` plus two other facts of your choice. Then write the starting facts a user would need to give for R7 to fire.

Part C: backward chaining. The goal is "*is a tiger?*". Which facts would the system need to check, and in what order? Which question would it ask the user if it knew only `gives milk`?

Part D: find the gap. A user enters `has feathers` and `can fly`. What does the system conclude? Is that a mistake by the computer, or a limit of the knowledge base? Explain in one or two sentences.

---

6. Expected outcomes and answers

Part A

Cycle	Working memory at start	Rule fired	New fact
1	gives milk, has hooves, has black stripes	**R2** (gives milk)	is a mammal
2	… + is a mammal	**R6** (is a mammal AND has hooves AND has black stripes)	is a zebra
3	… + is a zebra	none	**stop**

Conclusion: zebra, stopping in cycle 3. R5 (tiger) does **not** fire, because `eats meat` is not in working memory, even though two of its three conditions are true. AND needs *every* condition to be true (Topic 1). *You can check this answer by changing the starting facts in the example program.*

Part B (one possible answer)

```
R7: IF is a mammal AND has a very long neck AND has hooves THEN is a giraffe
Starting facts needed: gives milk, has a very long neck, has hooves
```

Any rule is acceptable if (1) it includes `is a mammal`, (2) the conditions are clear true/false facts, and (3) the starting facts include `gives milk`. Without it, R2 cannot produce `is a mammal`, and R7 can never fire. **Watch out:** if your giraffe rule used `has black stripes`, the same facts might also fire R6 and the system would conclude "zebra" too. Rules can conflict, which is why rule writers must check new rules against old ones.

Part C

1. The goal `is a tiger` is concluded only by **R5**, so the system needs `is a mammal`, `eats meat` and `has black stripes`.
2. `is a mammal` comes from **R2**, which needs `gives milk`. It is known ✔.
3. `eats meat` is not known, so the system would **ask: "Does the animal eat meat?"** (then "Does it have black stripes?").

The system asks only about facts that matter for the goal. It would never ask about feathers, because no step towards "tiger" needs them.

Part D

Cycle 1: R1 fires → `is a bird`. Cycle 2: no rule adds anything new → stop. The system concludes only **"is a bird"**, not which bird. R4 (owl) needs `hunts at night` as well, and no rule covers other flying birds.

This is **not** a computer error. The engine did exactly what the rules say. It is a **gap in the knowledge base**: the system knows only what its rules cover. This is the main weakness of rule-based systems, explored in Section 7 and in Topic 4.

Common mistakes

1. Firing a rule when only *some* of its AND conditions are true (e.g. firing R5 in Part A).
2. Firing the same rule again and again. A rule that adds nothing new does not count.
3. Forgetting that new facts (like `is a mammal`) can make other rules true in the *next* cycle.

---

7. Real expert systems, and their limits

System	When / where	What it did	Source
DENDRAL	From 1965, Stanford University (Feigenbaum and Lederberg)	Suggested the molecular structure of chemical compounds from spectrograph data; often called the first expert system	Britannica: Expert systems
MYCIN	From 1972, Stanford University	Diagnosed blood infections and recommended treatment using about 500 rules; could explain its reasoning; performed about as well as specialists and better than general practitioners	Britannica: Expert systems
XCON (R1)	Built 1978 by John McDermott (Carnegie Mellon) for Digital Equipment Corporation; in use from 1980	Chose the components for customers' computer orders; grew to about 2,500 rules; estimated to save the company about $25 million a year	Wikipedia: XCON

Strengths

- **Explainable:** every conclusion can be traced to named rules (Section 4.4).
- **Uses expert knowledge directly:** no large dataset is needed, only experts willing to state their rules.
- **Predictable:** the same facts always give the same answer.

Limits

- **No common sense, no awareness of its own limits.** Britannica gives a striking example: told that a patient was bleeding heavily from a gunshot wound, MYCIN would

still try to find a bacterial cause, because bacteria were all its rules knew about (Britannica: Expert systems).
- **Knows only what its rules cover**, as Part D showed. Anything unexpected is either missed or handled wrongly. Systems that fail like this outside their narrow area are often called **brittle**.
- **Hard to maintain.** Every new situation needs new rules, and new rules can conflict with old ones (Part B's warning). XCON needed thousands of rules.
- **Uncertainty is hard.** Real expert knowledge is rarely 100% certain, so real systems attached confidence or probability values to rules (Britannica: Expert system). Our simple true/false version leaves this out.

These limits are a large part of why interest in rule-only AI collapsed in the late 1980s (Topic 4), and why most modern AI learns from data instead (Topic 3).

---

8. Check your understanding

1. Which part of an expert system changes for every new case: the knowledge base or working memory?
2. In forward chaining, when does the inference engine stop?
3. A rule is `IF A AND B THEN C`. Working memory contains `A` only. Does the rule fire?
4. A doctor wants to test one suspected illness by asking the patient only relevant questions. Forward or backward chaining?
5. Give one reason why an expert system's answer may be easier to trust than an answer with no explanation.

Answers.

1. **Working memory.** The knowledge base (rules) stays the same.
2. When a full cycle passes in which **no rule can add a new fact**.
3. **No.** AND requires both A and B to be true.
4. **Backward chaining.** It starts from a goal and asks only for the facts needed to prove it.
5. It can show the exact rules it used, so a human expert can check each step (Section 4.4).

---

9. Summary

- **Symbolic AI** stores knowledge as human-readable **symbols** and **IF–THEN rules**. The computer does not understand the symbols; all meaning comes from the rules.
- A rule-based system has a **knowledge base** (rules), **working memory** (facts for this case) and an **inference engine** (the loop that fires rules).
- **Forward chaining** goes from facts to conclusions until nothing new can be added; **backward chaining** starts from a goal and works back to the facts.
- Rule-based systems can **explain** every conclusion, but they are **brittle**: they know only what their rules cover.

---

10. References

Links accessed 31 October 2026.

1. Encyclopaedia Britannica. *Expert system.* https://www.britannica.com/technology/expert-system
2. Encyclopaedia Britannica. *Artificial intelligence: Expert systems* (sections on knowledge base and inference engine, DENDRAL, MYCIN, and limitations). https://www.britannica.com/technology/artificial-intelligence/Expert-systems
3. Wikipedia. *Forward chaining* (definition; data-driven vs goal-driven; frog example). https://en.wikipedia.org/wiki/Forward_chaining
4. Wikipedia. *Xcon* (R1/XCON development, rule count, reported savings). https://en.wikipedia.org/wiki/Xcon

Further reading (not used for specific claims):

- Microsoft. *AI for Beginners, Lesson 2: Knowledge Representation and Expert Systems.* https://github.com/microsoft/AI-For-Beginners/blob/main/lessons/2-Symbolic/README.md
- Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.), Pearson. Chapters on logical agents and inference. *(Standard university textbook; team to confirm chapter numbers from the library copy.)*

Note on sources: Wikipedia is used only for the general definition of forward/backward chaining and for XCON figures, which are not covered in Britannica. Reviewers should confirm the XCON figures against a second source (see `docs/review-log.md`).

Topic 3: Symbolic AI vs Machine Learning

Level: Beginner | **Estimated time:** 30 minutes | **Status:** Drafted (awaiting team review)
Builds on: Topic 2: Rules and Inference | **Comes before:** Topic 4: History

Learning objectives. After this topic you will be able to:

1. Describe the difference between writing rules (symbolic AI) and learning from examples (machine learning).
2. Compare the two approaches on explainability, data needed, flexibility and maintenance.
3. Choose a suitable approach for a simple problem and justify the choice.

---

1. Two ways to give a computer knowledge

In Topic 2, **people** wrote every rule. The computer only applied them. This is sometimes called a **top-down** approach: knowledge flows from human experts down into the program.

There is a second way. Instead of writing the rules, you give the computer **many examples together with the correct answers**, and a program finds the patterns itself.

> **Machine learning (ML):** a branch of AI in which software learns patterns from data rather than following rules that a programmer wrote in advance (Britannica: Machine learning; IBM Developer: What is machine learning?).

> **Training:** the process in which an ML program studies example data to find patterns. **Model:** the result of training, the learned "rules", stored as numbers rather than as readable IF–THEN statements. **Labelled data:** examples that come with their correct answer, e.g. emails each marked "spam" or "not spam".

This is a **bottom-up** approach: knowledge is built up from the data.

Figure 3.1 Two ways to give a computer knowledge

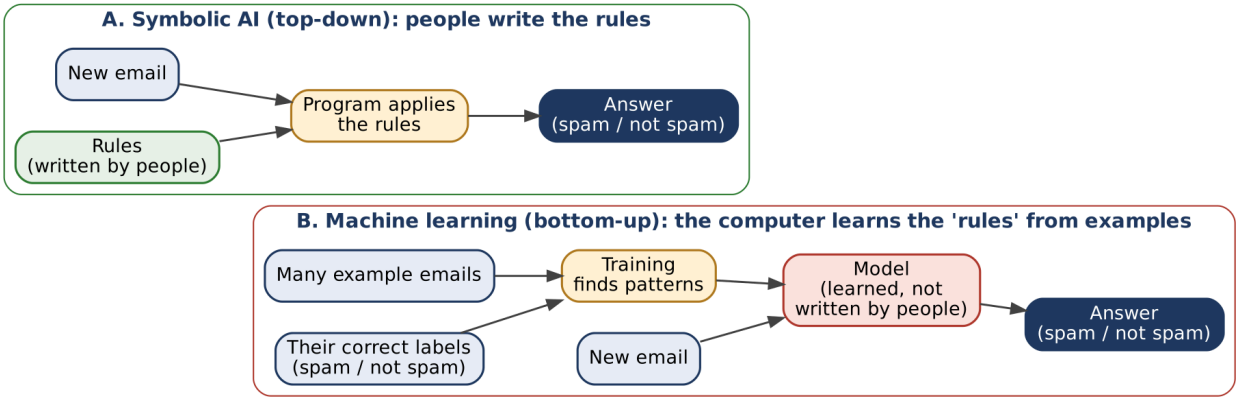


Figure 3.1. In symbolic AI, people supply the rules. In machine learning, people supply examples and answers, and the computer produces the model. Both end up answering new questions.

---

2. Same problem, two approaches: a spam filter

Symbolic version. A programmer writes rules:

```
R1: IF subject contains "FREE MONEY" THEN spam
R2: IF sender is in the user's contacts THEN not spam
R3: IF email contains more than 5 links AND sender is unknown THEN spam
```

- ✔ Easy to explain: "This was marked spam because of R1."
- ✘ Spammers quickly write "FR3E M0NEY", and R1 no longer matches. Someone must keep writing new rules.

Machine-learning version. The team collects, say, 100,000 emails that users have already marked as spam or not spam, and trains a model on them.

- ✔ The model picks up patterns no one thought to write down, and can be retrained on new spam.
- ✘ It needs a lot of labelled data. It is hard to explain exactly *why* a particular email was blocked, because the "rules" are thousands of learned numbers.

(These two filters are simplified teaching examples, not descriptions of a specific product. Real email services combine both approaches.)

---

3. Comparison

Question	Symbolic AI (rules)	Machine learning
Where does the knowledge come from?	Human experts write it as rules	Patterns found in example data
What does it need to get started?	Experts who can state their rules	Large amounts of (usually labelled) data
Can it explain an answer?	**Yes**: it lists the rules that fired (Topic 2, §4.4)	**Often difficult**: the model is many numbers
What happens with an unusual case?	Fails if no rule covers it ("brittle")	May handle it if similar examples were in the data; may also be confidently wrong
How is it updated?	People edit or add rules	Retrain on new data
Good fit	Clear, stable rules; answers must be justified (law, eligibility, safety checks)	Patterns too complex to write down (recognising faces, speech, handwriting)

Key idea: neither approach is "better" in general. Symbolic AI is strong where rules are clear and explanations matter; machine learning is strong where rules are too many or too vague to write by hand. Topic 5 shows that many modern systems use **both**.

---

4. Learner activity: which approach fits?

For each scenario, choose **symbolic AI**, **machine learning**, or **both**, and give one reason.

1. Checking whether a student meets the published entry requirements for a course (minimum grades in named subjects).
2. Recognising which animal is in a photo.
3. Calculating income tax from salary using the government's published tax bands.
4. Recommending songs based on what millions of users have listened to.
5. A hospital system that reads X-ray images **and** must flag any result that breaks a fixed safety guideline.

---

5. Expected outcomes and answers

#	Best fit	Reason
1	**Symbolic**	The requirements are explicit, published rules, and the decision must be explainable to the applicant.
2	**Machine learning**	No one can write IF–THEN rules for every possible photo of every animal; the patterns are learned from many labelled images.
3	**Symbolic**	Tax bands are exact rules set by law; using ML would add error and remove the explanation.
4	**Machine learning**	Taste patterns across millions of users are too complex to write down, and data is plentiful.
5	**Both**	ML is suited to reading images; the fixed guideline is a clear rule that should be checked by rules and explained.

Accept other answers if the reason refers to the comparison criteria in Section 3 (clear rules? explanation needed? enough data?).

---

6. Summary

- **Symbolic AI:** people write the rules (top-down); explainable, but brittle and hard to maintain.
- **Machine learning:** the computer learns patterns from labelled examples (bottom-up); flexible, but data-hungry and harder to explain.
- The right choice depends on whether the rules are clear, whether explanations are required, and whether enough data exists.

---

7. References

Links accessed 31 October 2026.

1. Encyclopaedia Britannica. *Machine learning.* https://www.britannica.com/technology/machine-learning
2. IBM Developer. *What is machine learning?* (contrasts rule-based programming with learning from data). https://developer.ibm.com/articles/what-is-machine-learning/
3. Encyclopaedia Britannica. *Artificial intelligence: Expert systems.* https://www.britannica.com/technology/artificial-intelligence/Expert-systems

Gaps for review: the spam-filter example is our own illustration and should be checked by a reviewer for accuracy. Section 3's comparison is a simplification; a reviewer should confirm that none of the rows is misleading for beginners.

Topic 4: History: The Rise, the AI Winters and the Lessons

Level: Beginner | **Estimated time:** 25 minutes | **Status:** Drafted (awaiting team review)
Builds on: Topic 2 and Topic 3 | **Comes before:** Topic 5: Symbolic AI in Modern Software

Learning objectives. After this topic you will be able to:

1. Place the main events in the history of symbolic AI on a timeline.
2. Explain what an "AI winter" is and give two causes of each of the two main winters.
3. Connect the second AI winter to the limits of rule-based systems you saw in Topic 2.

---

1. Key term

> **AI winter:** a period in which funding and interest in AI research fall sharply, usually after expectations were set too high and results did not match them (Wikipedia: AI winter).

The history of symbolic AI is a cycle that repeats twice: **big promises → funding → disappointment → winter**.

---

2. Timeline

Year(s)	Event	Phase
1956	Dartmouth summer workshop. John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon organise an eight-week meeting; the term "artificial intelligence" comes from this project (Dartmouth Alumni Magazine).	Optimism
1965	Work begins on DENDRAL, often called the first expert system (Britannica).	Optimism
1973	The **Lighthill report**, a review for the UK government, criticises AI for failing to meet its grand goals; UK AI research funding is largely dismantled.	First winter begins
1974	In the USA, DARPA cancels a major speech-recognition grant (about $3 million a year) as it moves towards mission-focused research.	First winter
≈1974–1980	**First AI winter.**	Winter
1980	XCON enters use at Digital Equipment Corporation (Topic 2, §7). Companies start building expert systems.	Boom
1981–1991	Japan's **Fifth Generation Computer** project (about $850 million) aims at computers that converse, translate and reason like people. By 1991 these goals are not met.	Boom → disappointment
1987	The market for specialised **LISP machines** (computers built to run AI software) collapses within about a year, because cheaper general-purpose computers become as powerful.	Second winter begins
≈1987–1993	**Second AI winter.** Expert systems prove expensive to maintain and brittle; US Strategic Computing funding for new AI is cut (1988).	Winter

Sources for 1973–1993 entries: Wikipedia: AI winter. Historians give slightly different start and end years; the ranges here are the commonly cited ones.

---

3. What went wrong?

First winter (≈1974–1980): promises ahead of computers

Early researchers believed general human-like intelligence was close. But the computers of the time were slow and had little memory, and problems such as understanding language turned out to be far harder than expected. When reviews like the Lighthill report compared the promises with the results, governments cut funding.

Second winter (≈1987–1993): the limits of rules

This is the part that connects directly to Topic 2. The expert-system boom of the 1980s ran into exactly the limits you met in the animal identifier:

Limit seen in Topic 2	What happened in industry
The system knows only what its rules cover (Activity Part D)	Systems failed on unusual cases; they were described as **brittle**
New rules can conflict with old ones (Activity Part B)	Large rule bases became very costly to update and maintain
No learning	Every change needed human experts and programmers

At the same time, the special hardware that AI companies sold (LISP machines) was overtaken by cheaper ordinary computers, and a whole industry disappeared within about a year (Wikipedia: AI winter).

The lesson

Rule-based AI did **not** disappear; it stopped being sold as a path to human-like intelligence and became an ordinary software technique (Topic 5). Meanwhile, research moved towards approaches that **learn from data** (Topic 3), which later benefited from much faster computers and much more data.

---

4. Learner activity

Part A. Put these events in order: Fifth Generation project starts · Dartmouth workshop · LISP machine market collapses · Lighthill report · XCON enters use.

Part B. For each winter, give **one** cause related to *expectations* and **one** cause related to *technology*.

Part C. In two sentences, explain how Activity Part D in Topic 2 (the "has feathers, can fly" case) helps explain the second AI winter.

---

5. Expected outcomes and answers

Part A. Dartmouth workshop (1956) → Lighthill report (1973) → XCON enters use (1980) → Fifth Generation project starts (1981/82) → LISP machine market collapses (1987).

Part B.

	Expectations	Technology
First winter	Researchers promised human-like intelligence soon; reviews (Lighthill, 1973) found results far short of this	Computers were too slow, with too little memory, for hard problems like language
Second winter	Expert systems and the Fifth Generation project promised more than they delivered	Rule bases were brittle and expensive to maintain; LISP machines were overtaken by cheaper computers

Part C (model answer). The system could only say "is a bird", because no rule covered flying birds that do not hunt at night; it failed on a case its builders had not foreseen. Real expert systems failed in the same way on a much larger scale, and the cost of writing ever more rules to fill such gaps helped end the expert-system boom.

---

6. References

Links accessed 31 October 2026.

1. Hertog, J. *The Birth of Artificial Intelligence.* Dartmouth Alumni Magazine. https://dartmouthalumnimagazine.com/articles/birth-artificial-intelligence
2. Encyclopaedia Britannica. *Artificial intelligence: Expert systems.* https://www.britannica.com/technology/artificial-intelligence/Expert-systems
3. Wikipedia. *AI winter.* https://en.wikipedia.org/wiki/AI_winter
4. Wikipedia. *Xcon.* https://en.wikipedia.org/wiki/Xcon

Gaps for review: several dates and amounts above come from Wikipedia only (Lighthill, DARPA grant, Fifth Generation budget, LISP machine collapse). Before this topic is marked **Checked**, a reviewer should confirm them in a second source, for example the Lighthill report itself or a textbook history chapter, and record this in `docs/review-log.md`.

Topic 5: Symbolic AI in Modern Software

Level: Beginner | **Estimated time:** 25 minutes | **Status:** Drafted (awaiting team review; module quiz is outline only) **Builds on:** Topics 1–4

Learning objectives. After this topic you will be able to:

1. Identify rule-based reasoning inside everyday software.
2. Explain what a knowledge graph is, using a simple example.
3. Describe how a "neuro-symbolic" system combines learning with rules, using one real example.

---

1. Rules did not disappear

Topic 4 ended with the AI winters. But the idea of IF–THEN rules plus an engine that applies them never went away. It became an ordinary part of software engineering, and is so ordinary that it is rarely called "AI" any more.

1.1 Business rules engines

> **Business rules engine:** software that stores an organisation's decision rules separately from the rest of the program, so that staff can update the rules without rewriting the program.

Salesforce, for example, sells a Business Rules Engine. Its own training material says that it automates complex decisions, organises rules in tools such as **decision tables**, and lists uses such as checking **eligibility** for discounts, fee refunds and public benefits, and calculating insurance quotes (Salesforce Trailhead: Get to Know Business Rules Engine).

> **Decision table:** a table in which each row is one rule: the columns on the left are conditions, the column on the right is the result. It is a compact way to write many IF–THEN rules.

Example: a student bus-pass discount as a decision table *(our own illustration)*

Rule	Is a student?	Age ≥ 65?	Result
D1	Yes	–	50% discount
D2	No	Yes	50% discount
D3	No	No	Full price

Compare this with Topic 1: each row is an IF–THEN rule, and the table as a whole is the OR condition `student OR age ≥ 65`. Compare it with Topic 2: the table is a small **knowledge**

base, and the rules engine is the **inference engine**.

1.2 Where else you meet rules *(examples for discussion; see review note)*

- **Online forms** that refuse to submit until required fields are filled in.
- **Tax and benefit calculators** that apply bands set by law (Topic 3, Activity Q3).
- **Characters in video games** that follow rules such as "IF the player is visible AND health is low THEN run away".

---

2. Knowledge graphs: facts as a network

> **Knowledge graph:** a large collection of facts stored as **things** (people, places, objects) connected by **relationships**, for example `Taj Mahal --located in--> Agra`.

In 2012 Google introduced its Knowledge Graph to help search understand "things, not strings". At launch it held over 500 million objects and more than 3.5 billion facts about them and their relationships. This lets a search tell, for example, the Taj Mahal monument apart from a musician of the same name (Google: Introducing the Knowledge Graph).

This is symbolic AI at a very large scale: facts are stored as readable symbols, and the system can follow the links between them, just as the inference engine in Topic 2 followed rules from fact to fact.

```
Taj Mahal --is a--> monument
Taj Mahal --located in--> Agra
Agra --located in--> India
=> Taj Mahal is located in India   (a new fact found by following two links)
```

---

3. Neuro-symbolic AI: learning and rules together

> **Neuro-symbolic AI:** a system that combines a **neural network** (a type of machine-learning model, Topic 3) with **symbolic reasoning** (rules and logic, Topic 2), so that each covers the other's weaknesses.

Case study: AlphaGeometry (Google DeepMind, January 2024) (DeepMind blog)

- **Symbolic part:** a deduction engine that applies geometry rules step by step to prove new statements, exactly like forward chaining.
- **Learning part:** a language model that, when the engine gets stuck, suggests one useful new construction (a point, line or circle) to add to the diagram. Then the engine continues.

- **Result:** on 30 olympiad geometry problems, it solved 25 within the time limit, compared with 10 for the previous best method. The average human gold medallist solves 25.9.
- **Why combine them:** DeepMind describes the language model as giving fast "intuition" and the symbolic engine as giving rigorous reasoning that can be checked.

This is a neat summary of the whole module: the rules provide reasoning that can be **explained and checked** (Topic 2), and learning provides **flexibility** that hand-written rules lack (Topic 3).

---

4. Learner activity: spot the rules

Choose an app or website you use often (for example a food-delivery app, a course-registration system, or a game).

1. Find **two** decisions it makes that are probably rule-based. Write each as an IF–THEN rule.
2. Find **one** feature that is probably based on machine learning. Explain why it would be hard to write as rules.
3. Is there a feature that seems to use **both**? Describe it in one sentence.

5. Expected outcome

There is no single correct answer; check that each response meets these criteria:

Part	A good answer…	Example (food-delivery app)
1	States clear true/false conditions and a result	`IF order total < minimum THEN disable "Checkout"`; `IF restaurant is closed THEN hide "Order now"`
2	Names a feature that depends on patterns in lots of data	"Recommended for you": depends on millions of past orders; too many patterns to write as rules
3	Shows a learned part plus a rule part	Delivery-time estimate is learned from past trips, **but** a rule adds 10 minutes IF it is raining (*hypothetical example*)

---

6. Module summary

Topic	Big idea
1	Computers decide by testing true/false conditions with IF–THEN rules
2	Symbolic AI = a knowledge base of rules + an inference engine that chains them, and it can explain itself
3	Machine learning learns patterns from data instead; each approach suits different problems
4	Over-promising and the limits of rules caused two AI winters
5	Rules live on in everyday software, knowledge graphs, and neuro-symbolic systems

Module self-assessment quiz (8 questions covering LO1–LO7): ***outline only****, planned for V2.0.*

---

7. References

Links accessed 31 October 2026.

1. Salesforce Trailhead. *Get to Know Business Rules Engine.* https://trailhead.salesforce.com/content/learn/modules/business-rules-engine/get-to-know-business-rules-engine
2. Singhal, A. (16 May 2012). *Introducing the Knowledge Graph: things, not strings.* Google. https://blog.google/products/search/introducing-knowledge-graph-things-not/
3. Google DeepMind (17 January 2024). *AlphaGeometry: An Olympiad-level AI system for geometry.* https://deepmind.google/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/

Review note: the examples in Section 1.2 and the activity's example answers are general illustrations, not claims about specific products. A reviewer should either find a source for each or keep them clearly labelled as illustrations.